

Parallel Crackme Challenge

Ahsan Ghafoor
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
ahgha0019@uit.no

I. INTRODUCTION

This report describes the design, implementation and evaluation of a brute-force parallel crackme challenge. Passwords are drawn from a 62-character set of digits and letters, and PBKDF2 derives the actual AES key from the password. We use MPI to transform a sequential solution into a parallel one.

A. Problem Definition

We have a set of 62 characters, of an unknown length. For each length of the password we have 62^n possible solutions, which leads to a huge number of passwords as the length grows. An example, a 4 character password is 62^4 that has over 14 million combinations, while for a 5 character password is over 900 million. If we try to brute force the password with a sequential program, this could take hours or days. To check if the password is correct we need to derive an AES key from it, attempt to decrypt the file with that key, and check whether it worked. If not, we need to try again with the next guess. This is a good problem to introduce parallelism. Instead of a single computer guessing over 14 million guesses, we have multiple computers each guessing a set of the possible solutions. One guess here is never dependent on the result of any other guess, which means we can check them in any order we want, by any number of workers, without needing to coordinate on the actual checking itself.

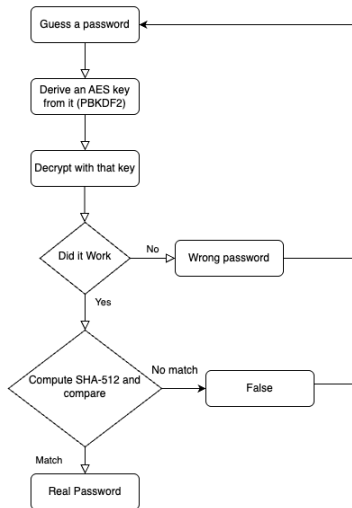


Fig. 1. The figure shows the password-verification flow, decryption is attempted first as a cheap rejection test, with the SHA-512 checksum only computed to confirm a successful decrypt.

B. Objectives

The objectives of this assignment are:

- Implement a sequential solution
- Implement an MPI parallel solution
- Implement an optimized MPI parallel solution
- Evaluate the results from the three solutions

C. Outline

The paper is organized as follows:

- Section II provides the relevant background.
- Section III explains the sequential code, design and how to execute
- Section IV explains how we parallelize the code, design and how to execute
- Section V explains how we optimize the parallel code, design and how to execute
- Section VI presents and analyses the results for the experiments done.
- Section VII discusses the findings.
- Section VIII discloses the use of AI tools.
- Section IX concludes the report.

II. BACKGROUND

A. Cluster

One of the most available distributed-memory systems is called a cluster [1, pp. 35]. A cluster is composed of a collection of commodity systems connected by a commodity interconnection network. Computer clusters are high performing and typically process data faster and handle larger workloads than a single computer [2]. For a brute force challenge, the extra performance makes a huge difference for how fast we can solve the problem. These benefits lead to powerful processors that can process massive data sets at an extremely high speed [2].

B. Parallel Programming

Parallel programming is a process where large compute problems are broken down into smaller problems that can be solved simultaneously by multiple processors [3]. Processes communicate by explicitly passing messages to each other. A type of parallel computing is data parallelism, by dividing data among the threads or processes [1, pp. 48]. This reduces serial time by running tasks concurrently.

C. MPI

One of the most used API for message-passing is the “Message-passing Interface” or MPI [1, pp. 54]. We use MPI to run programs across multiple nodes simultaneously by passing explicit messages between them. MPI provides speed, functionality designed for high performance and standardization across the industry [4].

III. SEQUENTIAL SOLUTION

The core algorithm we use is for each password length L , we iterate over index i from 0 to $62^L - 1$. A character is extracted by taking the index modulo the character set size, giving its position in the charset. The first character extracted ends up in the rightmost position of the password string. The index is then divided by 62, which strips off the digit we just extracted and leaves the remaining digits for the next step. This process repeats, moving one position to the left each time, so the string is built from right to left. Finally, the string is null-terminated.

An example is for $L = 3$ and $\text{index}=100$, where we start off with:

- $i=0$: $100 \% 62 = 38 = \text{“C”}$, $\text{idx} = 100/62 = 1$
- $i=1$: $1 \% 62 = 1 = \text{“1”}$, $\text{idx} = 1/62 = 0$
- $i=2$: $0 \% 62 = 0 = \text{“0”}$, $\text{idx} = 0$

Now we have computed “C, 1, 0”. As explained, the first character computed goes in the rightmost position, therefore C lands on position 3, 1 lands on position 2, and 0 lands on position 1, giving the password “01C”.

```
index_to_password(index, L, pwd):
    for (int pos = L - 1; pos >= 0; pos--)
    {
        pwd[pos] = CHARSET[index % 62];
        index = index / 62;
    }
    pwd[L] = '\0';
```

Listing 1. Converting an index to its corresponding password.

The per guess check is started first with a call to `index_to_password`, then `pbkdf2`, then `decrypt`, then `sha512sum`. The first turns the index to a password, then `pbkdf2` generates the AES-256 key from a given password, then we decrypt with AES-256, then we compute the SHA-512 checksum of the decrypted data, then we compare that computed checksum against the file’s known checksum, returning 0 if they match.

We have multiple points where we check. First is when we try to decrypt with the key derived from the password. This is a cheap rejection. If it passes then we compute the SHA-512 and compare. The checksum is more expensive, and only computed when needed. In some cases where the decrypt succeeds, the checksum will never match, since two unrelated inputs essentially never hash to the same SHA-512 value. In contrast, when decryption succeeds because the password is right, the checksum will always match.

Another fix was needed in how `sha512cmp` is used: it returns 0 on a match, but the precode’s condition used it

inverted. This would give a match when there was no match, and no match when there should be a match. Decrypt is also declared to return `uint32_t` but returns `-1` on failure. If we store that in an unsigned type instead of `int32_t`, that would change the number into a large positive value through modular arithmetic.

```
// load ciphertext and target checksum once
for L = 1, 2, 3, ...
    for i = 0, ..., 62^L - 1
        build pwd from i
        pbkdf2(pwd, L, key)
        plen = decrypt(ct, ct_len, key, pt)
        if plen >= 0
            sum = sha512sum(pt, plen)
            if sha512cmp(sum, target) == 0
                print pwd, stop // found
```

Listing 2. The per-guess check: decryption acts as a cheap rejection, with the SHA-512 checksum only computed and compared when decryption succeeds.

Another design choice worth mentioning is that buffers are sized to `MAX_FILE_SIZE_B` rather than something smaller. `file_load` checks the copy against `MAX_FILE_SIZE_B`, and not against the size of whatever buffer we gave it. If the buffer is smaller than the real file, then `file_load` writes past the end of it, into the next memory section. This results in a buffer overflow, which could lead to corrupted data or even a crash.

The second design choice worth mentioning is that timing starts after file loading rather than before. The file is loaded once, and in the case of strong scaling this would lead to a worse measured speedup compared to a “true” speedup of just the search. The speedup gets affected since the cost of file loading becomes a larger fraction of total time as the search time shrinks with more processes.

To run the solution, first run `make main_seq`. Then run the executable by `./main_seq ./files/<filename>`.

IV. PARALLEL SOLUTIONS

Before any searching starts, the encrypted file and its checksum are loaded once and distributed to every process. At first, every process independently called `file_load()`, relying on NFS to deliver data to each process. We changed this so that only rank 0 calls `file_load()`, then three `MPI_Bcast` calls distribute the data to every other rank. We broadcast first the ciphertext’s length, then the ciphertext bytes, and finally the checksum. The length must be broadcast before the ciphertext bytes, since every rank needs to know how many bytes to expect before that broadcast can happen.

For the parallel solution, we split the guess ranks across processes. This way, instead of 10 processes running 1000 guesses each, we have 10 processes each running only 100 guesses. We compute the start by taking the process rank, multiplying by the chunk size, and then adding the smaller of rank and remainder:

```
Start = Rank * chunk_size + (rank <
remainder ? rank : remainder)
```

We compute the end the same way as the start, but adding 1 to the rank first:

```
= (Rank + 1) * chunk_size + ((rank + 1)
< remainder ? (rank + 1) : remainder)
```

The remainder matters because we can't always split `chunk_size` evenly across all the processes. This leads to some processes having a bigger or smaller chunk size than the rest. Consider a naive split of 10 guesses across a size of 3: `chunk_size` would be 3, since integer division drops the remainder. This leaves index 9 with no home, and no process responsible for checking it. This is a correctness bug, not only a fairness one.

We use `MPI_Allreduce` for coordination. The function takes a send buffer, receive buffer, count, data type, operation, and communicator. `MPI_Allreduce` combines the values from every process using `MPI_MAX` and delivers the combined result back to all of them. We use a local found flag to stop a process's own inner loop, and a separate global found flag to control the outer loop. We can't use the local flag for the outer loop, since only the one process that found the password would have it set. `MPI_Allreduce` is collective, it waits for every process to participate. If the rank that found the password stopped and left the loop early, the remaining processes would call `MPI_Allreduce` again for the next length and wait forever for a process that is no longer there resulting in a deadlock.

```
// rank 0 loads ct and checksum, broadcasts both to
all ranks (MPI_Bcast)
```

```
for L = 1, 2, ... while !global_found
    n (num_guesses) = 62^L
    chunk = n / size
    rem (remainder) = n % size
    start = rank*chunk + min(rank, rem)
    end = (rank+1)*chunk + min(rank+1, rem)

    for i = start, ..., end-1, while !found
        build pwd from i
        pbkdf2(pwd, L, key)
        plen = decrypt(ct, ct_len, key, pt)
        if plen >= 0
            sum = sha512sum(pt, plen)
            if sha512cmp(sum, target) == 0
                found = 1
                print pwd
```

```
Allreduce(found, global_found, MAX)
```

Listing 3. The parallel search loop: each process searches its own fair share of the guess range for length `L`, then all processes synchronize once via `MPI_Allreduce` before moving to the next length.

To run the solution, first run `make mainMPI`. Then run the executable by `mpirun -np <N> ./mainMPI ./files/<filename>`.

V. OPTIMIZED PARALLEL SOLUTION

For the optimization, we try to terminate early with periodic checks mid-slice. This way we can catch the answer sooner, but with a trade-off between more frequent checks and communication overhead. We achieve this by calling `MPI_Allreduce` more often than once per length `L`, pausing every `check_interval` guesses to do a blocking `MPI_Allreduce` check. If the password is found, we can exit immediately.

We compute the checkpoint count from `chunk_size`, as it is the only value guaranteed identical across every process, rather than from each rank's own actual slice size, which can

differ by 1 due to the remainder. The consequence of basing this on something that isn't guaranteed identical across every process is that some processes would run more rounds than others, when the ones that finish early leave, the remaining processes would wait forever on an `Allreduce` call that will never be joined, resulting in a deadlock.

We also need to run the final "leftover round" and its `Allreduce` call unconditionally, even if a rank has zero guesses left to check. Skipping it for such a rank would again desync the `Allreduce` call count across ranks and risk a deadlock, and it also ensures every guess is checked, including the case where the solution lies in that last leftover range.

We use the global found flag, not the local found flag, to control the round loop. If a process finds the solution, it would otherwise make no more `Allreduce` calls, while the rest of the processes, having found nothing, would end up waiting for a process that has already exited, leading to a deadlock. The solution is that every process always shows up to every `Allreduce` call, only the process that already found the password stops doing new guesses, continuing to participate in `Allreduce` calls without guessing further until the global result confirms every process knows. The only place the local flag is used is in the innermost per guess loop.

For space, the listing uses shorthand:

- `n` = total guesses for this length (62^L)
- `chunk/rem` = the base chunk size and remainder from the partitioning formula
- `start/end` = this rank's guess range
- `chk` = number of checkpoint rounds
- `pos` = the guess currently being checked
- `r/r_end` = the current checkpoint round and its boundary
- `found/global_found` = the local and collectively-agreed match flags.

```
// rank 0 loads ct and checksum, broadcasts both to all
ranks (MPI_Bcast)
```

```
for L = 1, 2, ... while !global_found
    n = 62^L
    chunk = n / size
    rem = n % size
    start = rank*chunk + min(rank, rem)
    end = (rank+1)*chunk + min(rank+1, rem)
    chk = chunk / check_interval
    pos = start

    for r = 1, ..., chk, while !global_found
        r_end = pos + check_interval
        for pos = pos, ..., r_end-1, while !found
            // per-guess check (Listing 2)
            Allreduce(found, global_found, MAX)

        if !global_found
            for pos = pos, ..., end-1, while !found
                // per-guess check (Listing 2)
                Allreduce(found, global_found, MAX)
```

Listing 4. The optimized (v3) search loop: processes check in every `check_interval` guesses instead of once per length, catching an early find sooner at the cost of more frequent synchronization.

To run the solution, first run `make mainMPI_opt`. Then run the executable by `mpirun -np <N> ./mainMPI_opt ./files/<filename> [check_interval]`.

VI. PERFORMANCE EVALUATION

Local tests were run on a single machine (Apple M5, 10 cores, 16GB RAM). Cluster tests used 62 nodes from the department's shared cluster via a hostfile, predominantly a mix of Lenovo M910s desktops (Intel Core i7-7700 @ 3.60GHz, 4 cores, 32GB RAM) and HP EliteDesk 800 G5 SFF systems (Intel Core i7-9700 @ 3.00GHz, 8 cores, 16GB RAM), with a smaller number of other, more varied machines included as well.

Efficiency trends to a plateau as p grows, which is not the normal expectation for strong scaling. There is a drop from $p=1$ to $p=2$ (99.8% to 87.5%), but from $p=2$ through $p=62$ it remains consistently around 87%. As the number of processes grows, each process gets a smaller chunk of the total guesses, but the per-guess algorithm stays the same, it still needs to derive an AES key, attempt decryption, and compute and compare a SHA-512 checksum. In v2, there is only one synchronization point per password length L , regardless of the number of processes. This introduces a fixed synchronization overhead that neither improves nor worsens as more processes are added, within the range tested.

For v3, efficiency is worse than v2. This is the cost of the communication overhead built into the algorithm, and in the worst case this algorithm is affected more than the normal version. Here, v3 pays the full trade-off of the communication overhead without getting the benefit it is designed for.

TABLE I

STRONG SCALING RESULTS FOR A LENGTH-4 WORST-CASE PASSWORD ($T_1 = 158.44s$).

p	v2 (s)	v2 speedup	v2 eff.	v3 (s)	v3 speedup	v3 eff.
1	158.72	1.00	99.8%	159.20	1.00	99.5%
2	90.55	1.75	87.5%	90.32	1.75	87.7%
4	45.36	3.49	87.3%	45.64	3.47	86.8%
8	21.45	7.39	92.3%	21.79	7.27	90.9%
16	11.35	13.96	87.2%	11.43	13.87	86.7%
32	5.70	27.82	86.9%	5.72	27.69	86.5%
62	2.93	54.06	87.2%	3.00	52.90	85.3%

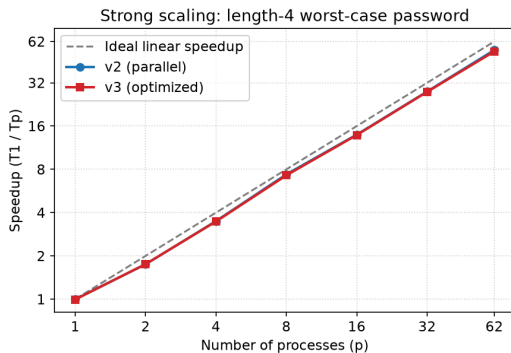


Fig. 2. Speedup vs. process count for v2 and v3, compared to ideal linear speedup (log-log scale).

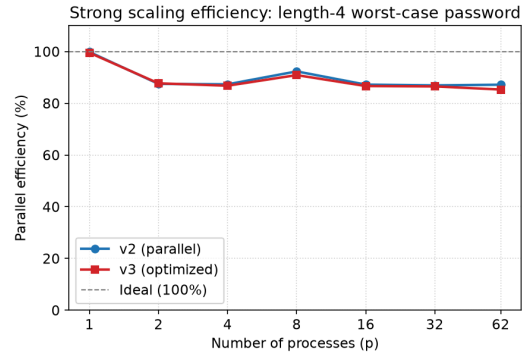


Fig. 3. Parallel efficiency vs. process count for v2 and v3.

Even though the work per process is held constant, time is not completely flat between $p=1$ and $p=62$ for either v2 or v3: time is higher at $p=62$ (2.92s) than at $p=1$ (2.57s), giving a weak-scaling efficiency of 87.7% for v2. v3 again performs worse than v2 here, for the same reason as in strong scaling, as weak scaling was also tested with the worst-case password, so v3 again loses its advantage. The jump from $p=1$ to $p=62$ is necessary because the search space size is 62^L : to keep the same workload per process, the number of processes must grow by exactly the same factor the search space grows by.

TABLE II

WEAK SCALING: 238,328 GUESSES PER PROCESS HELD CONSTANT AT BOTH POINTS.

Configuration	v2 time	v3 time
$p=1, L=3$	2.57s	2.57s
$p=62, L=4$	2.92s	3.02s
Weak-scaling eff.	87.7%	85.2%

Weak scaling: constant work per process (238,328 guesses)

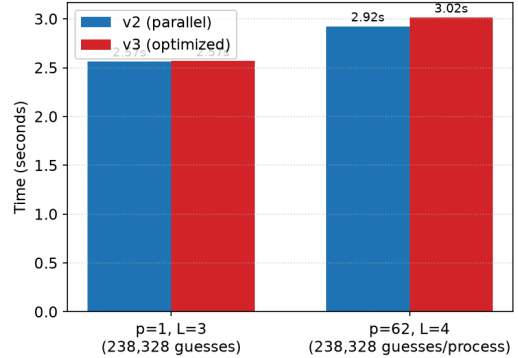


Fig. 4. Weak scaling: time at matched work-per-process (238,328 guesses) for v2 and v3.

Tests on the local machine with a worst-case password show time dropping from 160ms at `check_interval=50` to 128ms at `check_interval=1000`, converging toward v2's own baseline of 127ms. This shows that even in the worst case, v3 nearly matches v2 once the communication overhead is reduced.

TABLE III

CHECK_INTERVAL SENSITIVITY, LOCAL MACHINE, NP=4, LENGTH-3 WORST-CASE PASSWORD.

Version	Time (s)
v2 (no checkpointing)	0.1268
v3, check_interval=50	0.1601
v3, check_interval=100	0.1399
v3, check_interval=200	0.1288
v3, check_interval=500	0.1280
v3, check_interval=1000	0.1279

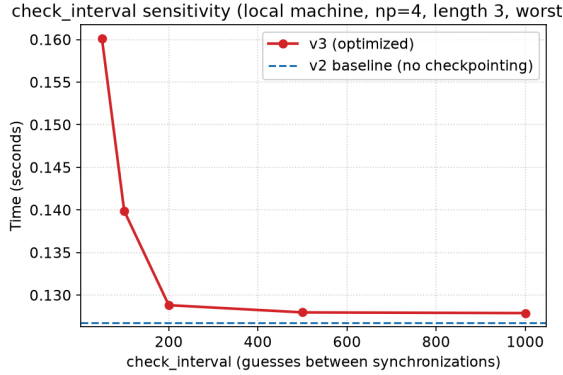


Fig. 5. check_interval sensitivity on the local machine (np=4, length 3, worst-case password).

The same test on the cluster shows a far larger effect. At check_interval=10, v2's baseline is about 3 times faster than v3 (9.41s vs 2.93s), but at large check_interval values, v3 again matches v2. The comparison between the two reveals how expensive a single MPI_Allreduce call is across many physically separate nodes, compared to within one machine: locally, the worst-case overhead is only around 26% (0.160/0.127), while on the cluster it is over 3x (9.41/2.93).

TABLE IV

CHECK_INTERVAL SENSITIVITY, CLUSTER, NP=62, LENGTH-4 WORST-CASE PASSWORD.

Version	Time (s)
v2 (no checkpointing)	2.9312
v3, check_interval=10	9.4112
v3, check_interval=25	4.9260
v3, check_interval=50	3.5365
v3, check_interval=100	3.1825
v3, check_interval=500	2.9857
v3, check_interval=1000	3.0129
v3, check_interval=5000	2.9302
v3, check_interval=10000	2.9295

check_interval sensitivity (cluster, np=62, worst-case password)

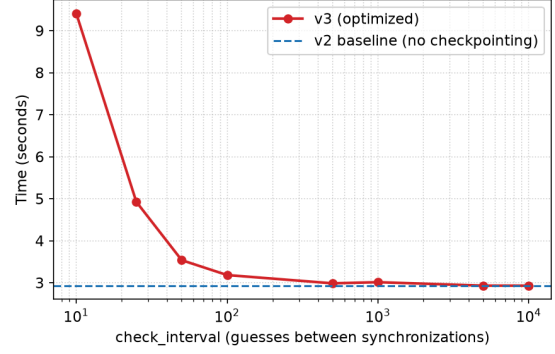


Fig. 6. check_interval sensitivity on the cluster (np=62, length 4, worst-case password).

The last test used a set of different, non-extreme passwords rather than only the worst case. v2 stays flat at 11.3-11.4s regardless of password, while v3 ranges from 0.30s (fastest) to 11.0s (slowest). This gives a mean speedup of 8.5x, but a median of only 2.3x. As explained earlier, v2 stays flat because it only synchronizes once per length, so every non-finding process always scans its entire slice no matter where the actual answer is. v3 varies wildly because its performance depends on where the password happens to fall relative to checkpoint boundaries, an early find, as with “Wc3J” or “Kx9F”, gives a huge speedup, while a late find, as with “7Bhy” or “mZ4x”, drags the mean down toward the median. The median, which reflects the speedup of a “typical” run, is what someone deciding whether to use v3 in practice should actually consider, not the mean, which is skewed upward by a small number of lucky outliers.

TABLE V

v3 SPEEDUP OVER v2 ACROSS 10 RANDOM LENGTH-4 PASSWORDS, NP=16. MEAN SPEEDUP: 8.5x. MEDIAN: 2.3x.

Password	v2 (s)	v3 (s)	Speedup
7Bhy	11.37	11.00	1.03x
mZ4x	11.37	10.67	1.07x
3mQa	11.32	9.97	1.14x
2xTk	11.33	7.58	1.49x
Q5vN	11.31	5.19	2.18x
9fQ2	11.34	4.56	2.49x
Tp7Z	11.41	3.55	3.22x
L8pR	11.36	2.04	5.56x
Wc3J	11.39	0.40	28.78x
Kx9F	11.34	0.30	38.15x

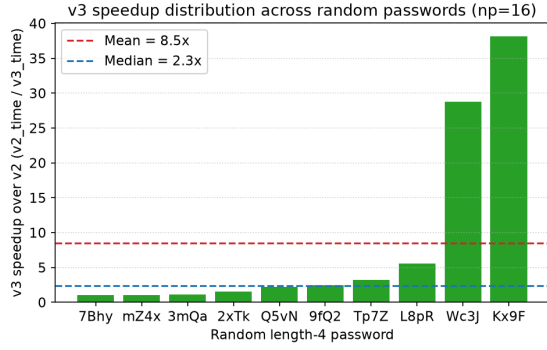


Fig. 7. v3 speedup distribution across 10 random length-4 passwords, with mean and median marked.

VII. DISCUSSION

One of the things that worked well was the partitioning, which is provably fair, as demonstrated in Section VI. We correctly found the deadlock and avoided it. Furthermore, strong scaling holds a flat 87% efficiency all the way to 62 processes, and in most cases v3 is faster, backed by real statistical evidence.

The evaluation also shows that v3 has a real communication overhead when there is no early find to exploit. The discrete 62x growth of the search space also makes “clean” weak scaling structurally awkward, since only one exact comparison point (a 62x jump in both p and problem size) was achievable. Finally, there is a large hardware gap between the local machine and the cluster, which complicates comparing results across the two directly.

Our 87% strong scaling efficiency compared to the ideal 100% perfect linear speedup comes from the fixed number of synchronization points, one Allreduce per length L, regardless of p. This is a good example of Amdahl’s law. The empirical mean/median results also match the heavy-tailed speedup theory.

In the future, we would move from the blocking approach to a non-blocking MPI_Iallreduce approach. Instead of pausing entirely while waiting for the reduction, a process could keep checking more of its own remaining guesses while the reduction completes in the background, only stopping once it later confirms the result.

We could also implement dynamic load balancing, instead of an idle process waiting once it has finished its own assigned range, the remaining guesses could be split among the processes still working, finishing the guess set faster as more processes contribute to what is left.

We could also implement a more adaptive or dynamic check_interval instead of a fixed one.

VIII. AI DECLARATION

AI was used throughout this project in a tutoring capacity for the code: concepts were explained and questions were asked to guide understanding, but all C and MPI code across all three versions was written, typed, and tested by the author.

For the report, AI reviewed drafts written by the author, pointing out unclear reasoning, and grammatical issues for the author to correct. All analysis, explanations, and conclusions in this report were written by the author, AI did not write the report’s explanatory or analytical content.

IX. CONCLUSION

We developed three versions of the parallel crackme challenge: one sequential, one parallel, and one optimized parallel solution. Through the performance evaluation, we confirmed correct and fair partitioning and a deadlock free coordination design, backed by real performance data gathered both locally and on the actual cluster. We found that v3 is not simply “faster”, its benefit depends heavily on where the password falls in the search space: dramatic in the best case, roughly break-even to worse in the worst case, with a real “typical” median speedup of 2.3x rather than 8.5x mean. This project demonstrates a concrete, measured trade-off in parallel programming: the cost of more frequent synchronization versus the benefit of stopping wasted work early, and shows that a correct optimization is not automatically a universally faster one.

REFERENCES

- [1] P. S. Pacheco, *An introduction to parallel programming*, 1st ed. Burlington, MA, USA: Morgan Kaufmann, 2011.
- [2] M. Flinders and I. Smalley, “What is cluster computing?.” Accessed: Sep. 11, 2026. [Online]. Available: <https://www.ibm.com/think/topics/cluster-computing>
- [3] M. Flinders and I. Smalley, “What is parallel computing?.” Accessed: Sep. 11, 2026. [Online]. Available: <https://www.ibm.com/think/topics/parallel-computing>
- [4] A. S. Gillis and S. J. Bigelow, “Message passing interface (MPI).” Accessed: Sep. 11, 2026. [Online]. Available: <https://www.techtarget.com/enterprise-software/definition/message-passing-interface-MPI>